left and right subtrees, with the caveat that we need to find the medians comparing the second coordinates for each point, instead. And so on and so forth for each level of the tree: we just need to keep track of the depth of the recursion, which tells us in what level of the tree we currently are.

The key point in listing 9.6 is the call to the `partition` method at line #4: we need to pass `level` as an argument because it will tell us which coordinate we need to use to compare points. The result of this call will be a tuple with the median of the `points` array and two new arrays with each, `(n-1)/2` elements, if `size(points) == n`.

Each point in `left` will be "smaller" (with respect to the coordinate at index `level % k`) than `median`, and each point in `right` will be larger than `median`: we can therefore recursively construct both (balanced) subtrees using these two sets.

Notice that we can't just sort the array once and chunk it up in halves recursively, because at each level the sorting criteria changes!

To understand how it works, let's consider a call to `constructKdTree([(0,5),(1,-1),(-1,6),(-0.5,0),(2,5),(2.5,3),(-1, 1),(-1.5,-2)])`.

The median for this set (WRT the first coordinate, i.e. the median of all the first values of the tuples) is either `-0.5` or `0`: there is an even number of elements, so there are technically 2 medians - you can double check the values by sorting the array.